

Codebase Vulnerability Scanner: C/C++ Vulnerability Detection Using Pretrained Code Models and Code Property Graphs

1 Introduction and Scope

Software vulnerabilities are mistakes in code that attackers can exploit. Researchers want to find these mistakes before software is put into use. This is especially important for language such as C and C++, where programmers directly manage memory. A mistake in a memory access or an interaction between functions can create a security risk. Finding these problems by hand takes time. Rule-based tools can help, but they may also raise false alarms: warnings about code that is not actually vulnerable. Machine-learning models offer another approach by learning patterns from examples of vulnerable and non-vulnerable code. However, their results depend heavily on the examples used for training and on how they are tested [1, 2].

The *Codebase Vulnerability Scanner* project focuses on *function-level detection*: deciding whether a C/C++ function contains a vulnerability. The project’s background covers three approaches: simple classifiers and models that read code in order, pretrained AI code models, and models that use graphs of program structure. A Code Property Graph (CPG) is a map that helps represent the structure of code, possible execution paths, and relationships between operations through graphs. This project will also test whether models work on software projects they have not seen during training. These ideas lead to a comparison of three systems: a simple baseline, a pretrained code model given further training for this task, and a hybrid that combines the same code model with CPG information.

1.1 Research Question and Sub-Problems

The high-level research question is:

How much will incorporating Code Property Graph-based program-structure information into a fine-tuned pretrained code model improve vulnerability detection in C/C++ code compared with a code-only model?

The question has four smaller parts:

1. How accurately will the simple classifier and pretrained models detect vulnerable C/C++ functions?
2. How will using a Code Property Graph(CPG) program structure improve the accuracy in vulnerability detection of the fine-tuned code model?
3. Will be including CPG make the model as effective even in previously unseen projects / codebase?

4. Will the accuracy of the encoder model + CPG depend on the type of vulnerability or be uniform?

2 General Review

2.1 General Review Map

The project’s background is organized around the following ideas:

- **How code is represented:** a model can receive word counts, code read in order, or a graph of relationships between code elements.
- **Learning from earlier training:** a pretrained model has already learned patterns from a large collection of code. Fine-tuning means giving it additional training for vulnerability detection.
- **Available datasets:** Juliet, Draper, Big-Vul, Devign, ReVeal, DiverseVul, and PrimeVul provide examples for training or testing.
- **Common models:** simple classifiers; convolutional neural networks (CNNs), which learn local patterns; long short-term memory networks (LSTMs), which read sequences; transformer code models, which relate tokens across the input; and graph neural networks (GNNs), which learn from connected code elements.
- **Main difficulties:** incorrect labels, repeated code, too few vulnerable examples, false alarms, and poor results on new projects. *Data leakage* occurs when information from the test set enters training or model selection, making results look better than they should.
- **Reference record:** Can be found towards the end as the Matrix record and Citations.

2.2 From Rule-Based Analysis to Learned Vulnerability Detection

Static analysis checks code without running it. Tools look for known risky patterns, trace data, or reason about possible execution paths. Their warnings are useful because they can point to a specific rule or program condition. Formal methods, such as bounded model checking, can check whether a program meets stated requirements within defined limits [3]. However, there may be too many paths to examine, and the tool may not fully model the program’s environment. To avoid missing problems, tools can also report possible risks that turn out to be harmless. In the industrial study by Du et al., developers commonly spent 10–20 minutes checking each warning. This motivates using learned models to help filter false alarms [4].

Early machine-learning systems described code using simple measurements, such as counts of words and symbols, then passed these measurements to a classifier such as logistic regression, a support-vector machine, or a random forest. These methods are inexpensive and provide a useful baseline. One example is TF-IDF, or "Term Frequency-Inverse Document Frequency", which gives more weight to terms that are frequent in one function but less common across the dataset. Used as a bag-of-words representation, it does not directly capture execution order. Two functions can contain the same terms but behave differently: checking a pointer before using it is not the same as checking it afterwards.

Sequence models improve on word counts by reading code elements such as names, keywords, and symbols in order. VUDENC uses Word2Vec to turn tokens into lists of numbers called embeddings,

then uses an LSTM to look for vulnerable regions [5]. It studies Python rather than C/C++, but its use of real projects and tests across projects is relevant here. Sequence models can learn patterns across nearby tokens. They can still struggle with relationships between distant operations, and they are not directly given a map of the program’s execution paths.

2.3 Pretrained Code Models

Pretrained code models have already learned from large amounts of source code before they are trained for a specific task. Rather than starting from scratch, researchers fine-tune them using examples labeled as vulnerable or non-vulnerable. Transformer models use *attention* to connect relevant tokens, including tokens that are far apart in the input. This helps them learn context, such as how a variable name or a function call is used. CodeBERT, GraphCodeBERT, CodeT5, and UniXcoder are examples used in vulnerability-detection research [2].

These models do not all learn from text alone. GraphCodeBERT also uses data-flow information during pretraining. It learns links between code tokens and graph nodes, and learns to predict graph connections [6]. Its results show that structure can help with general code tasks. However, this does not settle whether additional CPG information helps detect C/C++ vulnerabilities. Data flow alone may not describe the conditions that control an operation, calls between functions, or the order of several security-relevant steps.

CodeT5+ is another possible starting point. It learns through tasks such as filling in missing code, matching text with code, and predicting the next token [7]. Its encoder turns input code into a numerical representation that a classifier can use. However, CodeT5+ was designed for a broad range of code tasks, not specifically for C/C++ vulnerability detection. The project should choose its code model through small initial experiments and available computing resources, rather than assume that the strongest general-purpose model is best for this task.

The main advantage of a pretrained encoder is that it learns useful context from code. The concern is that a code-only input does not directly show all the relationships needed to judge safety. A model may recognize a familiar pattern without reliably determining whether a value reaches a risky operation or whether every path to that operation includes a safety check. Adding a graph gives the model these relationships more directly. It does not guarantee that the model will reason correctly, but it provides information that may help.

2.4 Graph-Based Program Representations

Program graphs are maps of relationships in code. Their nodes represent code elements, and their edges show how those elements are connected. The main types of information are:

- **Abstract Syntax Tree (AST):** the grammatical structure of code, such as which expression belongs to a statement.
- **Control-Flow Graph (CFG):** the order in which operations can run, like a flowchart with branches and loops.
- **Data-Flow Graph (DFG):** how values move from where they are created to where they are used.
- **Control dependence:** which conditions determine whether an operation runs. Program-dependence information includes both data and control dependencies.

Table 1: Representative approaches and their relevance to the Codebase Vulnerability Scanner project.

Approach	Representation	Relevance and limitation
TF-IDF + classifier	Weighted token counts	Simple, low-cost baseline, but does not directly show execution order or dependencies.
VUDENC [5]	Token embeddings + LSTM	Reads code in order and flags small regions, but targets Python and has no explicit program graph.
Pretrained encoder [7]	Code tokens in context	Reuses knowledge from earlier training, but may rely on familiar text patterns rather than program behavior.
AI4VA [9]	CPG + GGNN	Learns from syntax and dependencies, but results vary with the dataset and projects used for testing.
LineVD [10]	Transformer + dependency graph	Combines statement text with graph links, mainly to locate vulnerable statements.
DFEPT [11]	Pretrained model + DFG	Tests the benefit of adding data flow, but not a full CPG.

Yamaguchi et al. introduced the CPG to combine syntax, control flow, and program dependencies in one graph [8]. Their graph searches found several kinds of flaws, including previously unknown Linux kernel vulnerabilities. A CPG can therefore support questions such as: can an input reach a risky operation, and what checks lie along the way? The graph is still an approximation produced by analysis, not a guarantee that every possible runtime behavior has been captured.

Later studies used GNNs to learn from these graphs instead of relying only on hand-written graph searches. AI4VA turns C functions into CPGs and uses a gated graph neural network (GGNN) to predict whether each function is vulnerable [9]. The network repeatedly shares information between connected nodes. It outperformed static analyzers and several other learned models on two of three datasets. However, real-world code was harder than synthetic examples, and models built for individual projects could perform much better than models tested across projects. Graphs help, but they do not remove the need for realistic tests.

LineVD combines transformer representations of statements with a GNN that uses control and data dependencies [10]. The text representation describes a statement, while the graph shows its relationships with other statements. LineVD tries to identify vulnerable statements, not just vulnerable functions. This more detailed task is called *localization*. Although localization is outside the main scope of this project, LineVD shows why combining text and graph information is worth testing.

DFEPT provides a closer comparison. It builds data-flow graphs, learns a numerical representation of each graph, and joins that representation with the output of a pretrained code model [11]. Adding data flow generally improved F1, a score that balances precision and recall, for the tested CodeBERT, GraphCodeBERT, CodeT5, and UniXcoder combinations. The size of the gain depended on the model and dataset. Accuracy changed little or declined in some settings, so more graph information is not automatically better. Possible reasons include inaccurate graphs, repeated information, or a model learning training-specific patterns that do not transfer to new code.

2.5 Datasets, Leakage, and Generalization

The choice of training and test data can change the conclusions of a study. Synthetic datasets such as Juliet contain deliberately constructed examples with clear labels, but these examples may be simpler than real software. Other datasets collect functions from commits, or recorded code changes, that fix vulnerabilities. These are more realistic, but a fixing commit may also contain unrelated edits. This can lead to incorrect labels. Repeated code and almost-identical vulnerable and fixed versions can also end up in both training and testing, making the test easier than it should be.

DiverseVul provides a larger and more varied collection of real C/C++ code: 18,945 vulnerable and 330,492 non-vulnerable functions from 7,514 commits, covering 150 Common Weakness Enumeration (CWE) categories and hundreds of projects [12]. CWE categories identify types of software weaknesses. The study found that models still missed vulnerabilities, raised many false alarms, and struggled with rare weakness types. More training data did not always improve results, although more varied data could help on unseen projects. The large difference between the numbers of vulnerable and non-vulnerable functions is called *class imbalance*. It makes accuracy misleading: a model can label almost everything non-vulnerable and still appear accurate.

PrimeVul examines these testing problems more directly. Ding et al. show how repeated code, incorrect labels, and unrealistic data splits can overstate model performance [13]. PrimeVul uses stricter label checks, removes duplicates, and separates training and testing by date. Models that looked strong on earlier benchmarks performed much worse on this dataset. This warns against treating a high score on randomly split functions as proof that a model will work well in practice.

This project will therefore use *project-aware splits*: projects used for testing will not appear in training or validation. Where dates are available, a separate chronological test will train on older code and test on newer code. These answer different questions: working on unseen projects and working on later code. Exact and near duplicates will be removed before splitting. A random split can be included for comparison, but it will not be the main evidence of success.

Because vulnerable functions are rare, the evaluation will use more than accuracy:

- **Precision:** of the functions flagged as vulnerable, how many really are vulnerable?
- **Recall:** of all the vulnerable functions, how many does the model find?
- **F1 score:** a single score that balances precision and recall.
- **Matthews correlation coefficient (MCC):** an overall score that accounts for correct and incorrect predictions in both classes, making it useful when the classes are unequal.
- **Precision–recall area under the curve (PR-AUC):** a summary of the precision–recall trade-off across different decision thresholds, rather than at just one cutoff.

Results will also be broken down by CWE category where there are enough examples. Confidence intervals will show uncertainty in the reported scores.

2.6 Granularity, Explainability, and Practical Use

Studies differ in how precisely they identify a problem. Some label whole files or functions; others flag individual statements, lines, or selected groups of related statements called slices. This project starts with functions because they contain useful local relationships and match the labels in common datasets. However, saying that a function is vulnerable does not tell a developer which line to inspect. LineVD and VUDENC show the value of more detailed predictions [10, 5]. Finding the

exact location needs more detailed labels and will remain a separate extension.

For a tool to be useful, developers also need manageable numbers of false alarms and understandable warnings. Industrial evidence suggests that learned models can help filter warnings from static-analysis tools [4]. This does not mean that AI should replace static analysis or formal verification. The methods can support one another: analysis tools provide information about the program, and learned models help classify or prioritize it. The proposed hybrid follows this idea by giving a learned classifier both the code text and a graph built through static analysis.

3 Paper Review

3.1 Source-Selection Approach

Papers are chosen based on how closely they relate to this project, whether their papers or code are publicly available, and how clearly they explain their tests. Following the guideline, the reading process has three steps:

1. **Check relevance:** read the title, abstract, figures, and conclusion to decide whether the paper is useful for this project.
2. **Understand the study:** identify the problem, method, dataset, tests, and main results.
3. **Study the details:** look more closely at how the most relevant studies prepare their data, build their models, and test them. These include AI4VA, LineVD, DFEPT, DiverseVul, and PrimeVul.

The six selected papers cover models that read code in order, models that use graphs, models that combine both, and dataset quality. The following summaries give each paper’s details, main idea, data, methods, results, and lessons for this project.

3.2 AI4VA: Learning Vulnerability Signatures from CPGs

Identification. Suneja, Zheng, Zhuang, Laredo, and Morari (2020), “Learning to Map Source Code to Software Vulnerability Using Code-as-a-Graph.” No publisher DOI is listed; the public preprint is arXiv:2006.08614. A complete official implementation was not identified.

Main idea and research question. Can a model learn vulnerability patterns from the connections in a code graph? AI4VA converts each C function into a CPG, uses Word2Vec to turn node tokens into numerical features, and applies a GGNN to label the whole function as vulnerable or non-vulnerable [9].

Data, procedure, and techniques. The experiments use synthetic examples from s-bAbI and Juliet, plus real Debian and GitHub code from Draper. Names in the code are standardized so that the model relies less on project-specific naming. Learned embeddings describe node tokens, and the GGNN passes information along different types of graph edges. The comparisons include static analyzers, simple machine-learning models, CNNs, and recurrent neural networks (RNNs), which process sequences.

Metrics and findings. The paper reports F1 and average precision (AP), which summarizes precision as recall increases. AI4VA achieved F1/AP of 0.99/0.99 on s-bAbI, 0.87/0.96 on Juliet, and 0.50/0.40 on Draper. Results were much weaker on real code. Models trained for individual projects also performed better than a single model covering multiple projects. Graphs are useful, but difficult data and differences between projects remain important problems.

Key learning. CPGs are worth testing as an input to the proposed model. Strong results on synthetic examples are not enough; the model must also be tested on separate, real software projects.

3.3 VUDENC: Sequence Learning on Natural Repositories

Identification. Wartschinski, Noller, Vogel, Kehrler, and Grunske (2022), “VUDENC: Vulnerability Detection with Deep Learning on a Natural Codebase for Python.” DOI: 10.1016/j.infsof.2021.106809. The public code and data repository contains the scraping, preprocessing, embedding, and training pipeline.

Main idea and research question. Can an LSTM learn from real security fixes and point to suspicious regions of code? VUDENC learns from vulnerability-fixing commits and makes predictions over small groups of tokens [5]. It targets Python, but it offers useful lessons about reading code as a sequence and collecting training examples from real repositories.

Data, models, and procedure. The authors collect 1,009 security-fixing commits from GitHub covering seven types of web-related vulnerability. Word2Vec converts tokens into numerical representations. An LSTM then examines windows of nearby tokens and predicts whether they contain vulnerable code. The study tests different window sizes and evaluates the model both within projects and across projects.

Metrics and findings. Results range from 78–87% recall, 82–96% precision, and 80–90% F1, depending on the vulnerability type and model settings. The method also identifies suspected vulnerable regions and gives a confidence score for its predictions.

Key learning. Reading code in order can produce useful results, so a model without graph inputs is an important comparison. However, strong results on Python web vulnerabilities do not establish how well the same approach will find C/C++ memory-safety problems.

3.4 LineVD: Transformer and Dependency-Graph Fusion

Identification. Hin, Kan, Chen, and Babar (2022), “LineVD: Statement-Level Vulnerability Detection Using Graph Neural Networks.” DOI: 10.1145/3524842.3527949. The authors release source code and environment configuration.

Main idea and research question. Can combining statement text with code dependencies help a model find vulnerabilities and locate them more precisely? LineVD makes each statement a graph

node, uses CodeBERT to represent its text, and uses a GNN to share information between connected statements. Training labels tell the model which statements are considered vulnerable [10].

Data, procedure, and techniques. The study filters Big-Vul, which contains approximately 10,000 vulnerable and 177,000 non-vulnerable C/C++ functions. It uses removed lines in security fixes, together with lines that depend on added fixes, to create statement labels. The default split uses 80% of the data for training, 10% for validation, and 10% for testing. Training reduces the number of non-vulnerable functions, but validation and testing keep the class imbalance. The authors try randomly selected combinations of model settings, average results over ten runs, and use Wilcoxon signed-rank tests to assess whether improvements are statistically meaningful. They also run separate tests that hold out entire projects.

Metrics and findings. The study reports precision, recall, F1, and PR-AUC. It also uses ROC-AUC to summarize detection against false-positive rates, and ranking measures to check whether vulnerable statements appear near the top of the model’s suggestions. These include mean average precision at five (MAP@5), normalized discounted cumulative gain at five (NDCG@5), and mean first rank. Compared with IVDetect, LineVD increased statement-level F1 from 0.176 to 0.360, recall from 0.140 to 0.533, MAP@5 from 0.424 to 0.760, and PR-AUC from 0.520 to 0.642. Cross-project F1 was about 0.25–0.33 for the ten largest projects.

Key learning. Code text and graph connections can help each other. However, the default random split and labels inferred from fixes leave uncertainty about performance on new projects. This project will use the same general idea of combining inputs, but with stricter separation between training and test data.

3.5 DFEPT: Data-Flow Augmentation of Pretrained Models

Identification. Jiang et al. (2024), “DFEPT: Data Flow Embedding for Enhancing Pre-Trained Model Based Vulnerability Detection.” DOI: 10.1145/3671016.3671388. The public implementation includes fine-tuning scripts and configuration requirements.

Main idea and research question. Does giving a pretrained code model a separate description of data flow improve vulnerability detection? DFEPT builds a data-flow graph for each function and uses variable data types to describe its nodes. A graph model learns from these connections, with sinusoidal positional encoding to provide position information. The resulting graph representation is joined with a representation from CodeBERT, GraphCodeBERT, CodeT5, or UniXcoder [11].

Data, procedure, and techniques. The experiments use Devign and ReVeal. The authors compare different pretrained models, GNN designs, position information, and pooling methods. Pooling combines node information into one representation of the graph. Each hybrid is compared with the same pretrained model without DFEPT, helping show what the added data-flow information contributes.

Metrics and findings. The study reports accuracy, precision, recall, and F1. The best settings reached 64.97% accuracy on Devign and 47.9% F1 on ReVeal. Adding DFEPT improved F1 for most model combinations, but the gain depended on the dataset, pretrained model, and pooling method.

Key learning. DFEPT is the closest model design to this project’s proposal. It already tests the combination of pretrained code models and graph information. The next question is whether a fuller CPG, including syntax and control flow as well as dependencies, gives additional gains that hold up across projects.

3.6 DiverseVul: Scale and Diversity in C/C++ Data

Identification. Chen, Ding, Alowain, Chen, and Wagner (2023), “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection.” DOI: 10.1145/3607199.3607242. The project repository provides dataset access and evaluation resources.

Main idea and research question. Does a larger, more varied collection of real C/C++ code help models detect vulnerabilities? DiverseVul also asks whether broader testing reveals weaknesses that smaller datasets hide [12].

Data, procedure, and techniques. The authors collect security reports, find the commits that fix the reported problems, and extract functions before and after each fix. The dataset contains 18,945 vulnerable and 330,492 non-vulnerable functions from 7,514 commits, 150 CWE categories, and 348 projects. They train sequence, graph, and transformer models using DiverseVul alone and combined with earlier datasets. They also test performance on unseen projects.

Metrics and findings. The evaluation uses accuracy, precision, recall, and F1. Adding DiverseVul to earlier training data improved F1 for seven models by only 2.4% on average. It did not improve the strongest CodeT5 configuration. Models still had low F1 scores, raised many false alarms, and struggled with rare CWE categories.

Key learning. More varied data is useful, but simply adding examples does not solve incorrect labels, class imbalance, or poor results on new projects. This project can use DiverseVul as a main data source, provided it removes duplicates and keeps test projects separate.

3.7 PrimeVul: Realistic Evaluation of Code Models

Identification. Ding et al. (2025), “Vulnerability Detection with Code Language Models: How Far Are We?” DOI: 10.1109/ICSE55347.2025.00038. The official repository provides PrimeVul, preprocessing, trained-model resources, and evaluation code.

Main idea and research question. Do code models still perform well when labels are checked, duplicates are removed, and testing better reflects real use? PrimeVul provides both a dataset and a testing procedure to investigate this question [13].

Data, models, and procedure. PrimeVul brings together checked C/C++ vulnerability records. It contains 6,968 vulnerable and 228,800 non-vulnerable functions across 140 CWE categories. Code is standardized and duplicates are removed before training, validation, and testing are separated by date. The authors test pretrained encoders and larger code-generating models, including different prompts, class-balancing methods, and training approaches. They also test vulnerable functions alongside their fixed versions to see whether a model can recognize the security-relevant difference.

Metrics and findings. The study reports precision, recall, F1, performance on vulnerable/fixed pairs, and a vulnerability-detection score designed for imbalanced data. One model with seven billion parameters achieved 68.26% F1 on Big-Vul but only 3.09% on PrimeVul. Even larger models performed close to random behavior on strict paired tests. The large F1 difference shows how much the benchmark and evaluation conditions matter; it does not, by itself, measure how much of the difference is caused by leakage alone.

Key learning. PrimeVul is a central guide for this project’s testing procedure. A better model must show gains after duplicates are removed and data are properly separated. A high score on a random split is not enough to show that it will work on new code.

4 Literature Review Matrix

Table 2: Literature review matrix for the six selected studies.

Title, author, date	Conceptual framework	Research question	Dataset	Methodology	Analysis, results, conclusion	Future-research implication
AI4VA; Suneja et al.; 2020	Learn from code graphs	Can a GGNN find vulnerability patterns in CPG connections?	s-bAbI, Juliet, Draper	Build CPGs; describe nodes with Word2Vec; train a GGNN; compare models	F1 0.99/0.87/0.50 respectively; real code and tests across projects are much harder	Test graphs with pretrained code models on separate projects
VUDENC; Wartschinski et al.; 2022	Read real code in token order	Can an LSTM learn from fixes and locate vulnerable regions?	1,009 Python security-fixing commits; seven classes	Collect fixes; use Word2Vec and an LSTM on token windows	Precision 82–96%, recall 78–87%, F1 80–90%; results concern Python	Use real repositories for C/C++ and compare text-only with graph-based models
LineVD; Hin et al.; 2022	Combine statement text and graph links	Can text and dependencies help find vulnerable statements?	Filtered Big-Vul C/C++ functions	CodeBERT and GNN; 80:10:10 split; ten runs; separate held-out projects	F1 0.360 versus 0.176; PR-AUC 0.642; cross-project F1 about 0.25–0.33	Check labels carefully and prevent overlap between training and testing
DFEPT; Jiang et al.; 2024	Add data flow to pretrained code models	Does a separate DFG improve a code model’s predictions?	Devign and ReVeal	Describe DFG nodes by type; use a GNN and position information; join graph and code features	64.97% Devign accuracy; 47.9% ReVeal F1; gains vary with model and pooling	Test the full CPG and check which graph parts help
DiverseVul; Chen et al.; 2023	More varied real-world data	Does broader training data improve detection on familiar and new projects?	18,945 vulnerable and 330,492 non-vulnerable functions; 150 CWEs	Collect reports and fixes; extract functions; test several model types	Only 2.4% average F1 gain for seven models; imbalance and rare weaknesses remain difficult	Remove duplicates, check labels, and test on separate projects
PrimeVul; Ding et al.; 2025	Cleaner data and more realistic tests	Do models still perform well after data checks and date-based splitting?	6,968 vulnerable and 228,800 non-vulnerable functions; 140 CWEs	Check labels; standardize code; remove duplicates; test by date and on vulnerable/fixed pairs	A seven-billion-parameter model scored 68.26% F1 on Big-Vul but 3.09% on PrimeVul	Make careful data separation central; use paired tests to examine sensitivity to fixes

5 Synthesis and Research Gap

The selected papers provide four main lessons for this project:

1. Simple classifiers are useful baselines, but word counts miss important relationships in code.
2. Pretrained code models learn context, but good results on general code tasks do not guarantee good vulnerability detection.
3. Code graphs provide relationships that can help identify security problems.
4. Repeated data, incorrect labels, and testing on familiar projects can make models look more reliable than they are on new projects.

The remaining question for this project is: *does adding a CPG to the same fine-tuned code model improve function-level C/C++ vulnerability detection on unseen projects?* The selected studies do not fully settle this comparison. They use different datasets, predict at different levels, use only part of the graph information, or rely on random splits. DFEPT already combines pretrained models with data-flow graphs [11], so combining text and graphs is not itself a new idea. This project’s contribution is a controlled test of the added value of a fuller CPG, using the same data and stricter project separation.

The project will compare three systems, without assuming in advance that the more complex one is better:

1. **Simple baseline:** TF-IDF with logistic regression, using weighted counts of code tokens.
2. **Code-only model:** a pretrained encoder fine-tuned to classify functions, without a separate CPG input.
3. **Hybrid model:** the same code encoder combined with a learned CPG representation of syntax, control flow, and dependencies.

An *ablation study* will test which parts of the system matter by removing or changing them. Where computing resources allow, it will compare code-only, CPG-only, code+AST, code+CFG, code+DFG, and full code+CPG versions. This will show whether the graph helps, which relationships contribute, and whether the gains remain on unseen projects. Errors will also be examined by language, CWE category, function size, and the length of dependency chains.

Possible later extensions include selecting only security-relevant code, training directly on vulnerable/fixed pairs, locating vulnerable lines, and examining relationships across a whole repository. Keeping these outside the first experiment makes the main question easier to test: does giving the model a map of code structure help beyond giving it code text alone?

6 Project Solution Proposal

6.1 Planned Experimental Design

The project will combine PrimeVul and DiverseVul into one dataset, check labels, and remove repeated or near-identical functions, including overlap between the two sources. It will keep each function’s project, commit date, language, and CWE information where available. Juliet may provide an additional controlled test, but it will not replace testing on real code. All three main systems will use the same training, validation, and test functions. Training data teach the model, validation data guide model and setting choices, and test data measure final performance. Test projects will stay

separate from training and validation, and the final test set will remain unused until model selection is complete.

The hybrid will process each function in two ways. The code encoder will read its tokens and produce a numerical representation. A graph encoder will read its CPG, including node details and edge types, and pool the node information into a second representation. The system will join these two lists of numbers—a step called *concatenation*—and pass them to a small neural-network classifier, or multilayer perceptron. This provides a simple starting point before considering more complex ways to combine the inputs.

To address class imbalance, training will give more weight to errors on the less common vulnerable functions. Model settings will be compared using validation scores such as PR-AUC and F1. The decision threshold, which sets how high a vulnerability score must be before a function is flagged, will be chosen on the validation set using F1 or a stated false-positive limit. PR-AUC summarizes many thresholds, so it will not itself select a single cutoff. Each model will be trained several times with different random seeds to check whether results are stable. Paired bootstrap confidence intervals, obtained by repeatedly resampling the same test functions for both models, or a paired statistical test will help assess whether score differences are convincing.

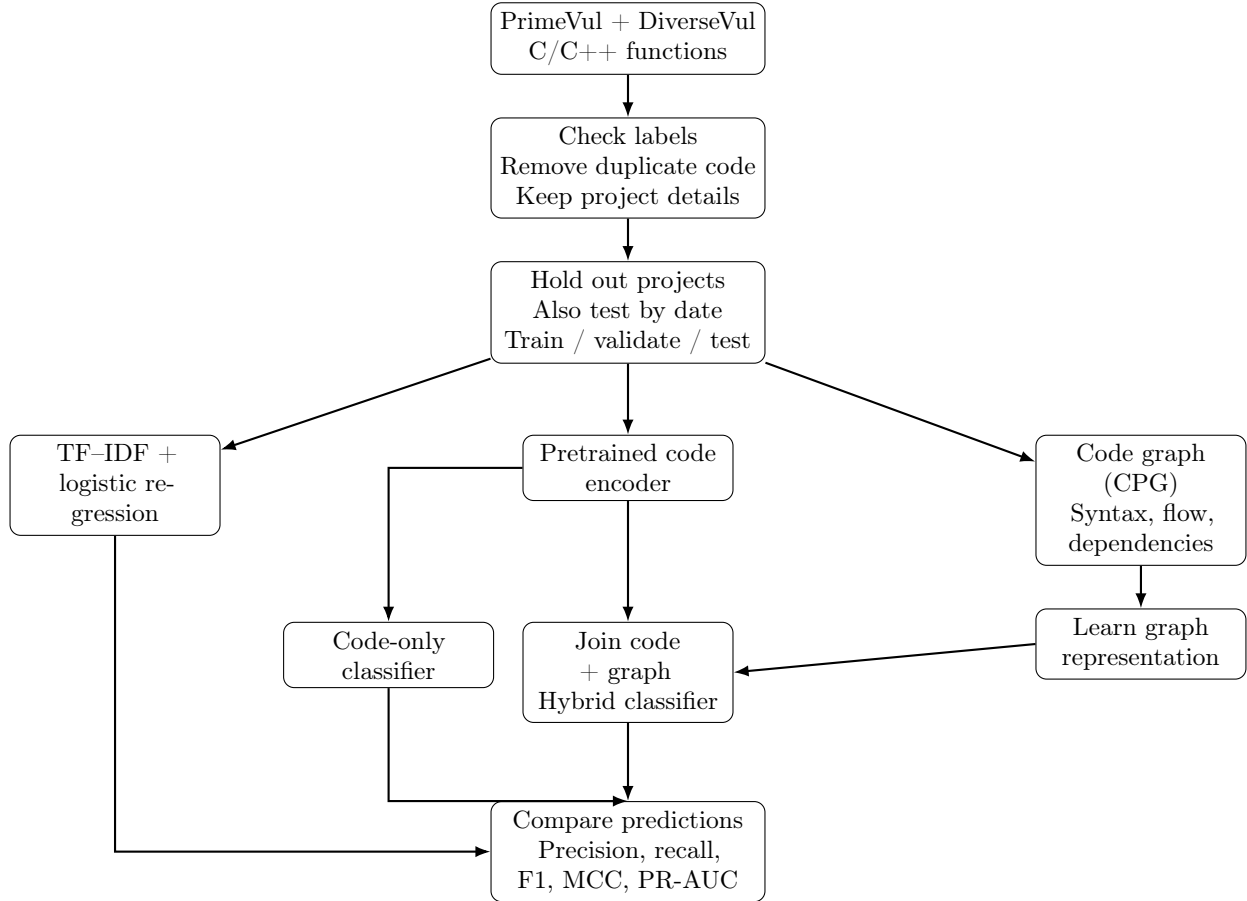


Figure 1: Tentative architecture and experimental comparison for the Codebase Vulnerability Scanner project.

The main experiment compares the code-only model with the hybrid on unseen projects. Additional

experiments compare random and project-aware splits, test later code where dates permit, report C and C++ separately, examine CWE categories, and remove graph components in turn. Success means more than higher accuracy: the hybrid must improve F1, MCC, or PR-AUC without an unacceptable increase in false alarms, using an acceptance rule set before final testing. Most importantly, the improvement must hold on projects that the model has not seen during training or validation.

References

- [1] N. Shiri Harzevili, A. B. Belle, J. Wang, S. Wang, Z. M. Jiang, and N. Nagappan, “A survey on automated software vulnerability detection using machine learning and deep learning,” arXiv:2306.11673, 2023.
- [2] F. Naseer, J. A. Khan, M. Yaqoob, A. Mylonas, and I. Gambo, “A systematic literature review for transformer-based software vulnerability detection,” arXiv:2604.24822, 2026.
- [3] P. Dantas, L. Cordeiro, and W. Junior, “ESBMC: A survey of its evolution, integration, and future directions in formal software verification,” arXiv:2605.26169, 2026.
- [4] X. Du et al., “Reducing false positives in static bug detection with LLMs: An empirical study in industry,” arXiv:2601.18844, 2026.
- [5] L. Wartschinski, Y. Noller, T. Vogel, T. Kehrner, and L. Grunske, “VUDENC: Vulnerability detection with deep learning on a natural codebase for Python,” *Information and Software Technology*, vol. 144, article 106809, 2022. doi: 10.1016/j.infsof.2021.106809.
- [6] D. Guo et al., “GraphCodeBERT: Pre-training code representations with data flow,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. Available: OpenReview.
- [7] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, “CodeT5+: Open code large language models for code understanding and generation,” in *Proceedings of EMNLP*, pp. 1069–1088, 2023. doi: 10.18653/v1/2023.emnlp-main.68.
- [8] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *Proceedings of the IEEE Symposium on Security and Privacy*, pp. 590–604, 2014. doi: 10.1109/SP.2014.44.
- [9] S. Suneja, Y. Zheng, Y. Zhuang, J. Laredo, and A. Morari, “Learning to map source code to software vulnerability using code-as-a-graph,” arXiv:2006.08614, 2020. Available: arXiv.
- [10] D. Hin, A. Kan, H. Chen, and M. A. Babar, “LineVD: Statement-level vulnerability detection using graph neural networks,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022. doi: 10.1145/3524842.3527949.
- [11] Z. Jiang, W. Sun, X. Gu, J. Wu, T. Wen, H. Hu, and M. Yan, “DFEPT: Data flow embedding for enhancing pre-trained model based vulnerability detection,” in *Proceedings of the 15th Asia-Pacific Symposium on Internetware*, 2024. doi: 10.1145/3671016.3671388.
- [12] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, “DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection,” in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, pp. 654–668, 2023. doi: 10.1145/3607199.3607242.

- [13] Y. Ding et al., “Vulnerability detection with code language models: How far are we?” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025. doi: 10.1109/ICSE55347.2025.00038.